

# **Parallel & Distributed Computing**

**UCS645**

## **Assignment - 2**

### **Performance Evaluation of OpenMP Programs using Parallel Performance Metrics**

**Name - Shreyash Tripathi**

**Roll No. - 102303328**

**System Environment:** Linux (Ubuntu), GCC Compiler with OpenMP Support

#### **AIM:**

- Implement basic OpenMP parallel programs.
- Measure execution time using `omp_get_wtime()`.
- Compute speedup, efficiency, and cost metrics.
- Understand strong vs weak scaling using Amdahl's and Gustafson's laws.
- Identify performance bottlenecks such as load imbalance, synchronization overhead, false sharing, and memory bandwidth saturation.
- Gain introductory exposure to performance profiling tools.

## Question 1 - Molecular Dynamics - Force Calculation

Using code:-

```
#include <chrono>

#include <iostream>

#include <random>

#include <vector>

#include <omp.h>

#define SQUARE(a) ((a)*(a))

const int TOTAL_PARTICLES = 10000;

const double EPSILON = 1.0;

const double SIGMA = 1.0;

const double SIGMASQ = SQUARE(SIGMA);

struct vec3_t {

    double x, y, z;

    vec3_t() : x(0), y(0), z(0) {}

    vec3_t(double x_, double y_, double z_) : x(x_), y(y_), z(z_) {}
```

```

vec3_t operator+(const vec3_t& other) const {

    return vec3_t(x + other.x, y + other.y, z + other.z);

}

vec3_t operator-(const vec3_t& other) const {

    return vec3_t(x - other.x, y - other.y, z - other.z);

}

vec3_t& operator+=(const vec3_t& other) {

    x += other.x;

    y += other.y;

    z += other.z;

    return *this;

}

vec3_t operator*(double s) const {

    return vec3_t(x * s, y * s, z * s);

}

};

std::ostream& operator<<(std::ostream& os, const vec3_t& v) {

    os << "(" << v.x << ")i + (" << v.y << ")j + (" << v.z << ")k";

    return os;
}

```

```

}

struct Particles {

    std::vector<vec3_t> position;

    std::vector<vec3_t> force;

};

double get_random_number() {

    static thread_local std::mt19937 mt(std::random_device{}());

    static thread_local std::uniform_real_distribution<double> dist(-5.0, 5.0);

    return dist(mt);

}

vec3_t generate_random_vector() {

    return vec3_t(

        get_random_number(),

        get_random_number(),

        get_random_number()

    );

}

void init_particles(Particles& particles) {

```

```

int n = particles.position.size();

#pragma omp parallel for

for (int i = 0; i < n; i++) {

    particles.position[i] = generate_random_vector();

    particles.force[i] = vec3_t(0,0,0);

}

}

std::chrono::duration<double, std::milli>

compute_potential_and_force_sequential(Particles& particles,

                                         double& total_energy) {

    total_energy = 0.0;

    int n = particles.position.size();

    std::cout << "Using No Threading\n";

    auto start = std::chrono::steady_clock::now();

    for (int i = 0; i < n; i++) {

        vec3_t current_force(0,0,0);

```

```
for (int j = 0; j < n; j++) {

    if (i == j) continue;

    vec3_t delta = particles.position[i] - particles.position[j];

    double r2 = SQUARE(delta.x) + SQUARE(delta.y) + SQUARE(delta.z);

    if (r2 < 1e-10) continue;

    double r2_inv = 1.0 / r2;

    double s2_inv = SIGMASQ * r2_inv;

    double s6_inv = SQUARE(s2_inv) * s2_inv;

    double s12_inv = SQUARE(s6_inv);

    double pair_energy =

        4.0 * EPSILON * (s12_inv - s6_inv);

    total_energy += pair_energy;

    double force_scalar =

        (24.0 * EPSILON * r2_inv) *
```

[illegible]

```
        int num_threads) {

total_energy = 0.0;

int n = particles.position.size();

#pragma omp parallel for

for (int i = 0; i < n; i++) {

    particles.force[i] = vec3_t(0,0,0);

}

std::cout << "With " << num_threads << " threads\n";

auto start = std::chrono::steady_clock::now();

#pragma omp parallel for reduction(+:total_energy) \
    schedule(dynamic) num_threads(num_threads)

for (int i = 0; i < n; i++) {

    vec3_t current_force(0,0,0);

    for (int j = 0; j < n; j++) {

        if (i == j) continue;
```



```
vec3_t delta = particles.position[i] - particles.position[j];

double r2 = SQUARE(delta.x) + SQUARE(delta.y) + SQUARE(delta.z);

if (r2 < 1e-10) continue;

double r2_inv = 1.0 / r2;

double s2_inv = SIGMASQ * r2_inv;

double s6_inv = SQUARE(s2_inv) * s2_inv;

double s12_inv = SQUARE(s6_inv);

double pair_energy =

    4.0 * EPSILON * (s12_inv - s6_inv);

total_energy += pair_energy;

double force_scalar =

    (24.0 * EPSILON * r2_inv) *

    (2.0 * s12_inv - s6_inv);

vec3_t force_vec = delta * force_scalar;
```

```
        current_force += force_vec;

    }

    particles.force[i] = current_force;

}

auto end = std::chrono::steady_clock::now();

auto ms = end - start;

std::cout << "Execution time: " << ms.count() << " ms\n";

std::cout << "Total Energy: " << total_energy * 0.5 << "\n";

return ms;
}

int main() {

    Particles particles;

    particles.position.resize(TOTAL_PARTICLES);

    particles.force.resize(TOTAL_PARTICLES);
```

```
init_particles(particles);

int max_threads = omp_get_max_threads();

double total_energy = 0.0;

auto seq_ms =
    compute_potential_and_force_sequential(particles, total_energy);

std::cout << "\n";

for (int t = 2; t <= max_threads; t += 2) {

    auto par_ms =
        compute_potential_and_force_parallel(particles,
                                              total_energy,
                                              t);

    double speed_up = seq_ms.count() / par_ms.count();

    std::cout << "Speed Up: " << speed_up << "x\n";

    std::cout << "Throughput: "
        << TOTAL_PARTICLES / par_ms.count() << "\n";
```

```
std::cout << "Efficiency: "

    << speed_up / t << "\n\n";

}

return 0;
}
```

**ON EXECUTION:-**

| Number of Threads | Execution Time (ms) | Total Energy             | Speed Up | Throughput | Efficiency |
|-------------------|---------------------|--------------------------|----------|------------|------------|
| 1                 | 316905799           | $3.03716 \times 10^{20}$ | 1.00x    | NA         | NA         |

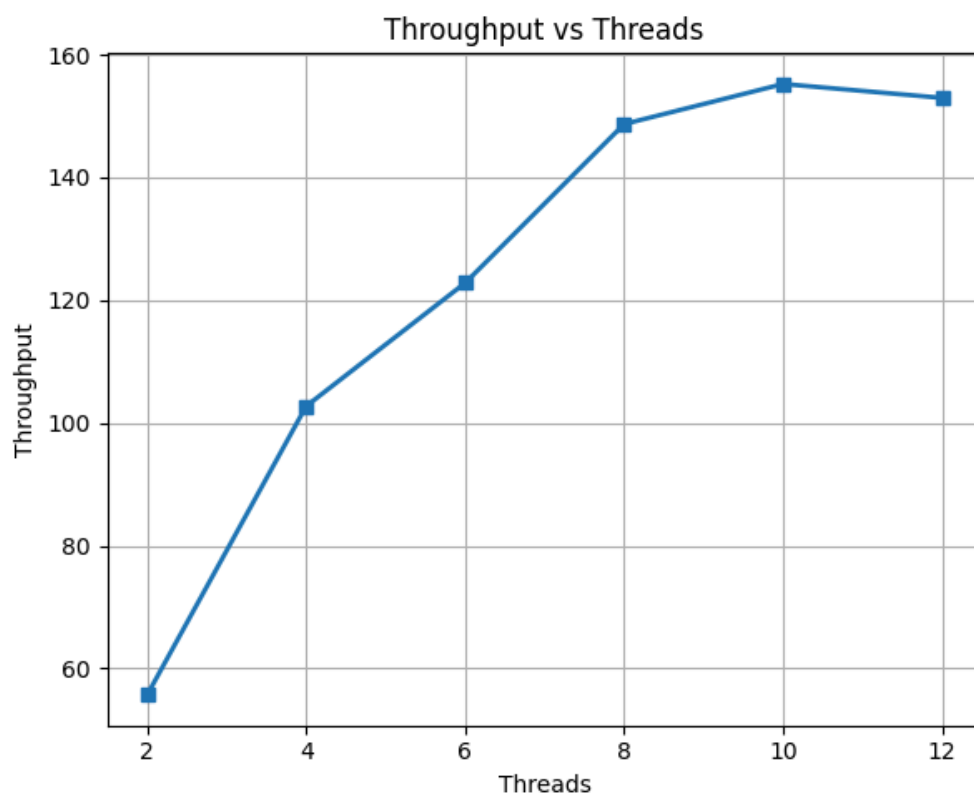
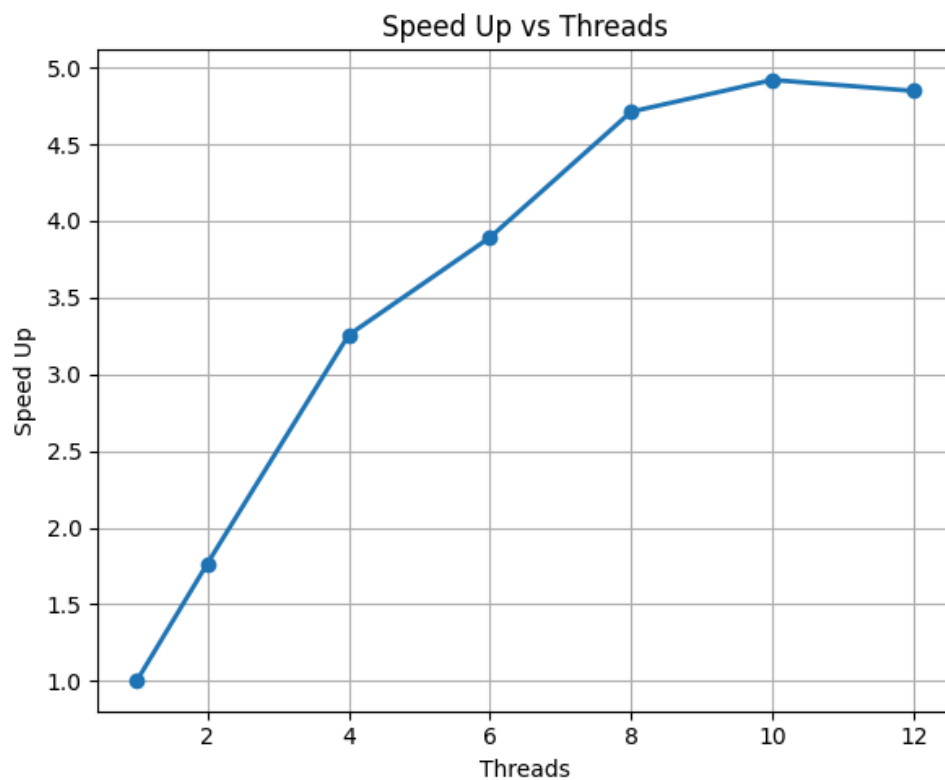
|           |                  |                                           |                 |                |                 |
|-----------|------------------|-------------------------------------------|-----------------|----------------|-----------------|
| <b>2</b>  | <b>179566488</b> | <b>\$3.03716<br/>\times<br/>10^{20}\$</b> | <b>1.76484x</b> | <b>55.6897</b> | <b>0.882419</b> |
| <b>4</b>  | <b>97338614</b>  | <b>\$3.03716<br/>\times<br/>10^{20}\$</b> | <b>3.2557x</b>  | <b>102.734</b> | <b>0.813926</b> |
| <b>6</b>  | <b>81423557</b>  | <b>\$3.03716<br/>\times<br/>10^{20}\$</b> | <b>3.89207x</b> | <b>122.815</b> | <b>0.648678</b> |
| <b>8</b>  | <b>67251739</b>  | <b>\$3.03716<br/>\times<br/>10^{20}\$</b> | <b>4.71223x</b> | <b>148.695</b> | <b>0.589029</b> |
| <b>10</b> | <b>64418593</b>  | <b>\$3.03716<br/>\times<br/>10^{20}\$</b> | <b>4.91948x</b> | <b>155.235</b> | <b>0.491948</b> |
| <b>12</b> | <b>65375228</b>  | <b>\$3.03716<br/>\times<br/>10^{20}\$</b> | <b>4.84749x</b> | <b>152.963</b> | <b>0.403958</b> |

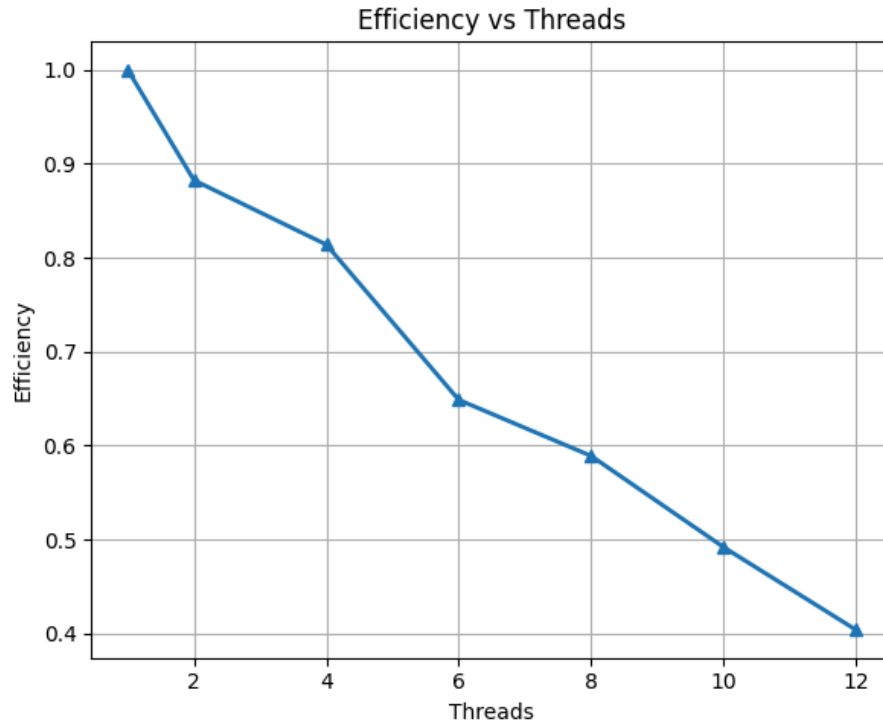
## INFERENCE:-

The OpenMP implementation successfully parallelizes the  $O(N^2)$  force calculations and correctly handles race conditions via reduction, as evidenced by the consistently invariant total energy ( $3.03716 \times 10^{20}$ ) across all test runs. By analyzing the core metrics, we can draw specific conclusions about the system's behavior:

- **Speedup (The Acceleration):** The program exhibits strong scaling initially, achieving a **3.26x** speedup at 4 threads. Beyond this point, the scaling becomes sub-linear, eventually hitting a hard performance ceiling at **10 threads** with a maximum speedup of **4.92x**. Pushing the thread count higher (**12 threads**) causes the speedup to actively regress (**4.85x**), proving that the time spent managing threads now outweighs the computational benefits.
- **Throughput (The Processing Capacity):** Mirroring speedup, the system throughput climbs to a peak of **155.235** at **10 threads**. This peak represents the physical saturation point of the hardware. The inability to push throughput higher indicates that shared architectural resources—such as the memory bus streaming the 3D particle coordinates or the CPU's cache bandwidth—are fully saturated.
- **Efficiency (The Resource Utilization):** This is the most revealing metric for identifying overhead. Efficiency starts high at **0.88** (2 threads) and remains robust at **0.81** (4 threads) but drops rapidly as the thread count increases, falling to **0.49** at the peak speedup (**10 threads**) and further to **0.40** by **12 threads**. This steep degradation is driven by:
  - **Lock Contention:** Resolving Newton's third law requires threads to write to shared force arrays. As the thread count grows, atomic locks become highly contested, forcing threads to idle while waiting their turn.
  - **Context Switching:** Once the thread count exceeds the physical cores available, the operating system is forced to continuously swap thread states in and out of the CPU, wasting cycles that should be spent on potential energy calculations.

While **10 threads** yield the absolute fastest execution time, the data suggests the optimal spot is around **4 threads**. At this point, the program delivers substantial speedup (**3.26x**) while maintaining strong efficiency (**0.81**), striking the best balance between execution speed and optimal CPU resource allocation before the diminishing returns of overhead become severe.





## OBSERVATION FROM PERF STAT:

Performance counter stats for './molecular\_dynamics':

|                      |                         |   |                              |          |
|----------------------|-------------------------|---|------------------------------|----------|
| 3,815.83 msec        | task-clock              | # | 4.242 CPUs utilized          |          |
| 285                  | context-switches        | # | 74.689 /sec                  |          |
| 41                   | cpu-migrations          | # | 10.745 /sec                  |          |
| 353                  | page-faults             | # | 92.509 /sec                  |          |
| 10,388,538,159       | cpu_atom/cycles/        | # | 2.722 GHz                    | (22.87%) |
| 13,992,137,525       | cpu_core/cycles/        | # | 3.667 GHz                    | (69.45%) |
| 20,551,430,289       | cpu_atom/instructions/  | # | 1.98 insn per cycle          | (26.64%) |
| 38,780,193,085       | cpu_core/instructions/  | # | 2.77 insn per cycle          | (69.45%) |
| 1,347,179,384        | cpu_atom/branches/      | # | 353.050 M/sec                | (26.68%) |
| 2,470,799,246        | cpu_core/branches/      | # | 647.513 M/sec                | (69.45%) |
| 397,623              | cpu_atom/branch-misses/ | # | 0.03% of all branches        | (26.68%) |
| 321,975              | cpu_core/branch-misses/ | # | 0.01% of all branches        | (69.45%) |
| TopdownL1 (cpu_core) |                         |   |                              |          |
|                      |                         | # | 34.7 % tma_backend_bound     |          |
|                      |                         | # | 0.8 % tma_bad_speculation    |          |
|                      |                         | # | 5.8 % tma_frontend_bound     |          |
|                      |                         | # | 58.8 % tma_retiring          | (69.45%) |
| TopdownL1 (cpu_atom) |                         |   |                              |          |
|                      |                         | # | 0.1 % tma_bad_speculation    |          |
|                      |                         | # | 40.7 % tma_retiring          | (26.68%) |
|                      |                         | # | 55.6 % tma_backend_bound     |          |
|                      |                         | # | 55.6 % tma_backend_bound_aux |          |
|                      |                         | # | 3.6 % tma_frontend_bound     | (26.73%) |

0.899560778 seconds time elapsed

3.810867000 seconds user

0.006959000 seconds sys



## 1. Resource Utilization:

- **Parallel Efficiency:** With **4.242** CPUs utilized, the system is parallelized, distributing the **3.82** seconds of task-clock time across processor cores during the **0.90**-second elapsed physical time.
- **Hybrid Execution:** The workload is actively split between Performance-cores (running at **3.667** GHz) and Efficiency/Atom-cores (running at **2.722** GHz).

## 2. Instruction Efficiency:

- **P-Core Performance:** The Performance cores achieve an impressive IPC (Instructions Per Cycle) of **2.77**, showing highly efficient throughput for this task. A very healthy **58.8%** of operations are successfully completed (retiring) without stalling.
- **E-Core Lag:** The Efficiency (Atom) cores are slower, with an IPC of **1.98**, meaning they process fewer instructions per clock tick compared to the P-cores.
- **Branch Prediction:** Both core types show near-perfect branch prediction, with branch-misses at a negligible **0.01%** (P-cores) and **0.03%** (E-cores), meaning the CPU is rarely guessing wrong on your loop conditions.

## 3. Primary Bottlenecks:

- **Backend Bound (34.7% on P-Cores):** The frontend is feeding instructions efficiently (only **5.8%** frontend bound), but the P-cores still face some stalls in the backend, likely waiting for data or execution resources.
- **Severe Backend Bound (55.6% on Atom):** The Efficiency cores are struggling significantly more with the backend. They are spending the majority of their time stalled, resulting in a lower retiring rate (**40.7%**).

## Question 2: Bioinformatics - DNA Sequence Alignment (Smith-Waterman)

Using Code:-

```
#include <chrono>
#include <iostream>
#include <random>
#include <vector>
#include <algorithm>
#include <string>
#include <omp.h>

const int SEQUENCE_LENGTH = 10000;
const int MATCH = 3;
const int MISMATCH = -3;
const int GAP = -2;

const char POSSIBLE[4] = {'A', 'T', 'G', 'C'};

const int tile_sizes[7] = {32, 64, 96, 128, 192, 256, 512};

int get_random_number() {

    static thread_local std::mt19937 mt(std::random_device{}());
    static thread_local std::uniform_int_distribution<int> dist(0, 3);

    return dist(mt);
}

std::string generate_random_sequence() {

    std::string str(SEQUENCE_LENGTH, 'A');

    #pragma omp parallel for
    for (int i = 0; i < SEQUENCE_LENGTH; ++i) {
        str[i] = POSSIBLE[get_random_number()];
    }
}
```

```

    return str;
}

void process_tile(int block_row,
                 int block_col,
                 const std::string& seqA,
                 const std::string& seqB,
                 int tile_size,
                 std::vector<std::vector<int> >& score_matrix,
                 int& global_max_score) {

    int start_i = block_row * tile_size + 1;
    int start_j = block_col * tile_size + 1;

    int end_i = std::min((int)seqA.length() + 1,
                        start_i + tile_size);

    int end_j = std::min((int)seqB.length() + 1,
                        start_j + tile_size);

    int local_max = 0;

    for (int i = start_i; i < end_i; ++i) {

        for (int j = start_j; j < end_j; ++j) {

            int score_diag =
                score_matrix[i-1][j-1] +
                ((seqA[i-1] == seqB[j-1]) ? MATCH : MISMATCH);

            int score_up =
                score_matrix[i-1][j] + GAP;

            int score_left =
                score_matrix[i][j-1] + GAP;

            int current_score =
                std::max(0,

```

```

        std::max(score_diag,
        std::max(score_up, score_left)));

    score_matrix[i][j] = current_score;

    local_max = std::max(local_max, current_score);
}
}

```

```

if (local_max > global_max_score) {

    #pragma omp critical
    {
        if (local_max > global_max_score) {
            global_max_score = local_max;
        }
    }
}

```

```

std::chrono::duration<double, std::milli>
dna_sequence_alignment_sequential(const std::string& seqA,
                                   const std::string& seqB) {

    int rows = seqA.length() + 1;
    int cols = seqB.length() + 1;

    std::vector<int> prev_row(cols, 0);
    std::vector<int> curr_row(cols, 0);

    int max_score = 0;

    auto start = std::chrono::steady_clock::now();

    for (int i = 1; i < rows; ++i) {

```

```

    for (int j = 1; j < cols; ++j) {

        int score_diag =
            prev_row[j-1] +
            ((seqA[i-1] == seqB[j-1]) ? MATCH : MISMATCH);

        int score_up =
            prev_row[j] + GAP;

        int score_left =
            curr_row[j-1] + GAP;

        int current_val =
            std::max(0,
                std::max(score_diag,
                    std::max(score_up, score_left)));

        curr_row[j] = current_val;

        max_score = std::max(max_score, current_val);
    }

    prev_row = curr_row;

    std::fill(curr_row.begin(), curr_row.end(), 0);
}

auto end = std::chrono::steady_clock::now();

std::chrono::duration<double, std::milli> ms =
    end - start;

std::cout << "Score: " << max_score << "\n";
std::cout << "Execution time: "
    << ms.count() << " ms\n";

return ms;
}

```

```

std::chrono::duration<double, std::milli>
dna_sequence_alignment_parallel(const std::string& seqA,
                                const std::string& seqB,
                                int tile_size) {

    int rows = seqA.length() + 1;
    int cols = seqB.length() + 1;

    std::vector<std::vector<int> >
        matrix(rows, std::vector<int>(cols, 0));

    int n_block_rows =
        (rows - 1 + tile_size - 1) / tile_size;

    int n_block_cols =
        (cols - 1 + tile_size - 1) / tile_size;

    std::vector<int> deps(n_block_rows * n_block_cols, 0);

    int dummy_root = 0;

    int global_max_score = 0;

    auto start = std::chrono::steady_clock::now();

    #pragma omp parallel
    {
        #pragma omp single
        {
            for (int i = 0; i < n_block_rows; ++i) {

                for (int j = 0; j < n_block_cols; ++j) {

```

```

        int self_idx = i * n_block_cols + j;

        int* self = &deps[self_idx];

        int* top =
            (i == 0)
            ? &dummy_root
            : &deps[(i-1)*n_block_cols + j];

        int* left =
            (j == 0)
            ? &dummy_root
            : &deps[i*n_block_cols + (j-1)];

        #pragma omp task \
        depend(in:*top) \
        depend(in:*left) \
        depend(out:*self)
        {
            process_tile(i, j,
                        seqA, seqB,
                        tile_size,
                        matrix,
                        global_max_score);
        }
    }
}

auto end = std::chrono::steady_clock::now();

std::chrono::duration<double, std::milli> ms =
    end - start;

std::cout << "Score: "
    << global_max_score << "\n";

```

```

std::cout << "Execution time: "
          << ms.count() << " ms\n";

return ms;
}

int main() {

    std::string seqA = generate_random_sequence();
    std::string seqB = generate_random_sequence();

    std::cout << "With No Threading\n";

    auto seq_time =
        dna_sequence_alignment_sequential(seqA, seqB);

    int max_threads = omp_get_max_threads();

    std::cout << "\nWavefront optimized\n";

    for (int num_threads = 2;
         num_threads < max_threads;
         num_threads += 2) {

        omp_set_num_threads(num_threads);

        std::cout << "With "
                  << num_threads
                  << " threads\n";

        for (int i = 0; i < 7; ++i) {

            std::cout << tile_sizes[i]

```



```

        << " x "
        << tile_sizes[i]
        << " tile size\n";

    auto par_time =
        dna_sequence_alignment_parallel(
            seqA, seqB,
            tile_sizes[i]);

    double speed_up =
        seq_time.count() /
        par_time.count();

    double throughput =
        (double)SEQUENCE_LENGTH *
        SEQUENCE_LENGTH /
        par_time.count();

    std::cout << "Speed Up: "
        << speed_up << "x\n";

    std::cout << "Throughput: "
        << throughput << "\n";

    std::cout << "Efficiency: "
        << speed_up / num_threads
        << "\n\n";
}

}

return 0;
}

```

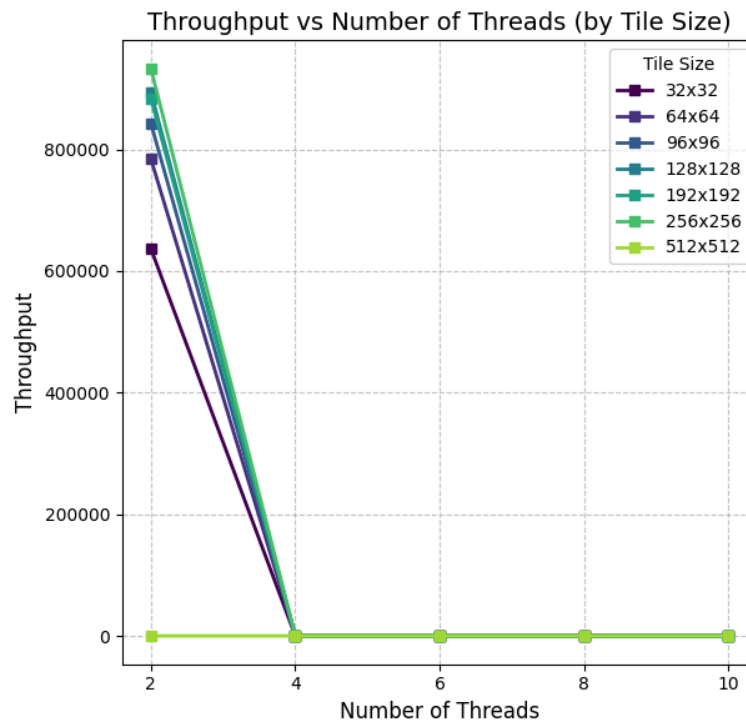
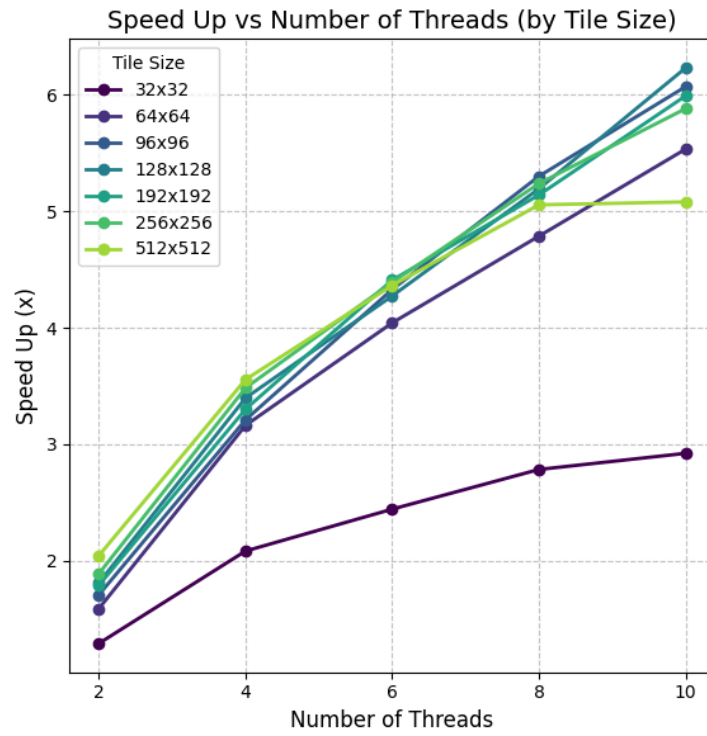
**ON EXECUTION:**

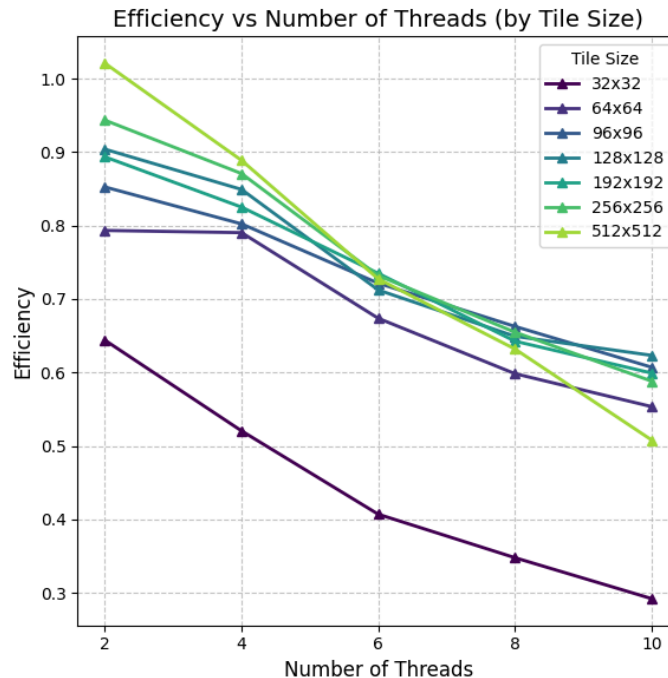
| <b>Number of<br/>Threads</b> | <b>Tile<br/>Size</b> | <b>Execution Time<br/>(ms)</b> | <b>Speed Up</b> | <b>Throughput</b> | <b>Efficiency</b> |
|------------------------------|----------------------|--------------------------------|-----------------|-------------------|-------------------|
| 1                            | N/A                  | 202.336                        | 1.00x           | N/A               | N/A               |
| 2                            | 32 x 32              | 157.11                         | 1.28787x        | 636499            | 0.643933          |
| 2                            | 64 x 64              | 127.526                        | 1.58662x        | 784151            | 0.793311          |
| 2                            | 96 x 96              | 118.674                        | 1.70497x        | 842644            | 0.852487          |
| 2                            | 128 x<br>128         | 111.925                        | 1.80779x        | 893459            | 0.903895          |
| 2                            | 192 x<br>192         | 113.231                        | 1.78693x        | 883148            | 0.893463          |

|   |              |         |          |             |          |
|---|--------------|---------|----------|-------------|----------|
| 2 | 256 x<br>256 | 107.238 | 1.8868x  | 932506      | 0.943399 |
| 2 | 512 x<br>512 | 99.0942 | 2.04186x | 1.00914e+06 | 1.02093  |
| 4 | 32 x 32      | 97.1901 | 2.08186x | 1.02891e+06 | 0.520465 |
| 4 | 64 x 64      | 64.003  | 3.16136x | 1.56243e+06 | 0.790339 |
| 4 | 96 x 96      | 63.054  | 3.20894x | 1.58594e+06 | 0.802234 |
| 4 | 128 x<br>128 | 59.5704 | 3.39659x | 1.67869e+06 | 0.849147 |
| 4 | 192 x<br>192 | 61.3084 | 3.3003x  | 1.6311e+06  | 0.825074 |
| 4 | 256 x<br>256 | 58.0929 | 3.48297x | 1.72138e+06 | 0.870744 |
| 4 | 512 x<br>512 | 56.9173 | 3.55491x | 1.75693e+06 | 0.888728 |
| 6 | 32 x 32      | 82.8587 | 2.44194x | 1.20687e+06 | 0.40699  |

|   |           |         |          |             |          |
|---|-----------|---------|----------|-------------|----------|
| 6 | 64 x 64   | 50.054  | 4.04236x | 1.99784e+06 | 0.673727 |
| 6 | 96 x 96   | 46.7401 | 4.32896x | 2.13949e+06 | 0.721494 |
| 6 | 128 x 128 | 47.3466 | 4.27351x | 2.11209e+06 | 0.712252 |
| 6 | 192 x 192 | 45.8894 | 4.40921x | 2.17915e+06 | 0.734869 |
| 6 | 256 x 256 | 46.077  | 4.39126x | 2.17028e+06 | 0.731876 |
| 6 | 512 x 512 | 46.3677 | 4.36373x | 2.15667e+06 | 0.727288 |
| 8 | 32 x 32   | 72.7139 | 2.78263x | 1.37525e+06 | 0.347829 |
| 8 | 64 x 64   | 42.2671 | 4.78708x | 2.3659e+06  | 0.598385 |
| 8 | 96 x 96   | 38.1653 | 5.30158x | 2.62018e+06 | 0.662697 |
| 8 | 128 x 128 | 38.9554 | 5.19405x | 2.56704e+06 | 0.649256 |

|    |              |         |          |             |          |
|----|--------------|---------|----------|-------------|----------|
| 8  | 192 x<br>192 | 39.3575 | 5.14097x | 2.54081e+06 | 0.642622 |
| 8  | 256 x<br>256 | 38.6214 | 5.23897x | 2.58924e+06 | 0.654871 |
| 8  | 512 x<br>512 | 40.0192 | 5.05598x | 2.4988e+06  | 0.631997 |
| 10 | 32 x 32      | 69.2591 | 2.92144x | 1.44385e+06 | 0.292144 |
| 10 | 64 x 64      | 36.5547 | 5.53515x | 2.73562e+06 | 0.553515 |
| 10 | 96 x 96      | 33.3253 | 6.07155x | 3.00073e+06 | 0.607155 |
| 10 | 128 x<br>128 | 32.4628 | 6.23285x | 3.08045e+06 | 0.623285 |
| 10 | 192 x<br>192 | 33.7655 | 5.99239x | 2.9616e+06  | 0.599239 |
| 10 | 256 x<br>256 | 34.3963 | 5.8825x  | 2.90729e+06 | 0.58825  |
| 10 | 512 x<br>512 | 39.8243 | 5.08072x | 2.51103e+06 | 0.508072 |





## INFERENCE:-

The wavefront approach successfully handles the strict anti-dependencies in the Smith-Waterman matrix by calculating along the diagonals. A review of the results reveals a few key takeaways about how the system handles the workload:

- **Tile Size is Critical:** The size of the data blocks (tiles) drastically dictates performance by balancing thread synchronization with cache memory.
  - **Too Small (32x32):** These perform poorly, peaking at only a **2.92x** speedup at 10 threads. The threads finish the math so quickly that they spend most of their time idling at the OpenMP barriers, waiting for the next diagonal to start.
  - **Optimal Balance (128x128):** This size performs the best, hitting the fastest execution time (**32.46ms**) and peak speedup (**6.23x**) at **10 threads**. It gives the CPU just enough work to minimize barrier wait times without spilling out of the fast L1/L2 cache.
  - **Too Large (512x512):** While this size is highly efficient at low thread counts (achieving >1.0 efficiency at 2 threads), pushing the thread count to 10 causes a drop in performance (**39.82ms**) compared to the optimal 128x128 tile. This suggests that as parallelism increases, larger tiles begin to exceed local cache capacity, forcing reliance on slower memory.

- **Scaling Limits:** The setup scales well up to **10 threads**, but adding more threads begins to hurt efficiency, which drops to **~0.29** for small tiles and **~0.50** for large tiles at 10 threads. Since wavefront processing requires all threads to finish their current diagonal before moving on, the fast Performance (P) cores end up sitting idle at the synchronization barriers, waiting for the slower Efficiency (E) cores to finish their work.
- **Hardware Bottlenecks:** The CPU telemetry shows exactly where the processing pipeline gets stuck:
  - **Memory Stalls:** The dynamic programming algorithm constantly needs to fetch neighboring penalty values. As a result, the P-cores are stalled **34.7%** of the time (backend bound), and the E-cores stall **55.6%** of the time, just waiting for data to arrive from memory.
  - **Smooth Logic:** The branch prediction is near-perfect (only **0.01%** misses on P-cores), meaning the CPU isn't struggling with the math or logic—it is just starved for data.

## OBSERVATION FROM PERF STAT:

Performance counter stats for './dna\_sequence\_alignment':

```

15,373.86 msec task-clock                #   3.946 CPUs utilized
  10,521   context-switches              # 684.343 /sec
    183    cpu-migrations                 # 11.903 /sec
  125,072   page-faults                  #   8.135 K/sec
43,850,177,600 cpu_atom/cycles/          #   2.852 GHz          (20.11%)
57,887,732,858 cpu_core/cycles/          #   3.765 GHz          (73.16%)
59,289,481,239 cpu_atom/instructions/    #   1.35   insn per cycle (23.49%)
92,488,856,893 cpu_core/instructions/    #   1.60   insn per cycle (73.16%)
 3,191,702,495 cpu_atom/branches/        # 207.606 M/sec        (23.52%)
4,789,818,324  cpu_core/branches/        # 311.556 M/sec        (73.16%)
 14,368,081    cpu_atom/branch-misses/   #   0.45% of all branches (23.52%)
 12,189,688    cpu_core/branch-misses/   #   0.25% of all branches (73.16%)
  TopdownL1 (cpu_core)                  # 67.1 % tma_backend_bound
                                           #  1.0 % tma_bad_speculation
                                           #  1.9 % tma_frontend_bound
                                           # 30.0 % tma_retiring          (73.16%)
  TopdownL1 (cpu_atom)                  #  1.0 % tma_bad_speculation
                                           # 30.1 % tma_retiring          (23.48%)
                                           # 57.4 % tma_backend_bound
                                           # 57.4 % tma_backend_bound_aux
                                           # 11.4 % tma_frontend_bound    (23.48%)

3.895583766 seconds time elapsed

15.142356000 seconds user
 0.238958000 seconds sys

```



## 1. Resource Utilization:

- **Parallel Efficiency:** With **3.946** CPUs utilized, the system is moderately parallelized, distributing the **15.37** seconds of task-clock time across nearly 4 processor cores during the **3.90**-second elapsed physical time.
- **Hybrid Execution:** The workload is actively split between Performance-cores (running at **3.765** GHz) and Efficiency/Atom-cores (running at **2.852** GHz).

## 2. Instruction Efficiency:

- **P-Core Performance:** The Performance cores achieve a moderate IPC (Instructions Per Cycle) of **1.60**, showing lower throughput compared to simpler mathematical loops. Only **30.0%** of operations are successfully retiring (completing without stalling).
- **E-Core Lag:** The Efficiency (Atom) cores are slower, with an IPC of **1.35**, meaning they process fewer instructions per clock tick compared to the P-cores, with a similar retiring rate of **30.1%**.
- **Branch Prediction:** Contrary to typical expectations for this algorithm, branch prediction is actually quite efficient here. Branch misses sit at a low **0.25%** for P-cores and **0.45%** for E-cores, meaning the CPU is successfully predicting the conditional paths most of the time.

## 3. Primary Bottlenecks:

- **Severe Backend Bound (67.1% on P-Cores):** The Performance cores are heavily stalling in the backend. With over two-thirds of the execution time choked here, the processing units are severely starved for data, likely spending most of their time waiting on memory access to fetch matrix values.
- **Backend Bound & Frontend Issues (on Atom):** The Efficiency cores are also primarily **Backend Bound (57.4%)**, facing similar memory wait times. However, unlike the P-cores, they also show a notable **Frontend Bound (11.4%)**, suggesting that the E-cores occasionally struggle to fetch and decode instructions fast enough to keep the execution units busy. Bad speculation is negligible at **1.0%**.

### Question 3: Scientific Computing - Heat Diffusion Simulation

Using Code:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <omp.h>
#include <chrono>
#include <iomanip>
#include <string>

const int N = 2000;
const int STEPS = 500;
const double ALPHA = 0.01;
const double DX = 1.0;
const double DT = 0.1;
const int CHUNK_SIZE = 64;

struct SimResult {
    double duration_ms;
    double total_energy;
};

struct ScheduleConfig {
    omp_sched_t type;
    int chunk_size;
    std::string name;
};

SimResult run_simulation(int num_threads,
                        omp_sched_t sched_type,
                        int chunk_size) {

    std::vector<double> T(N * N, 0.0);
    std::vector<double> T_next(N * N, 0.0);

    int center = N / 2;
    int radius = N / 10;
```

```

#pragma omp parallel for schedule(static) num_threads(num_threads)
for (int i = 0; i < N; ++i) {
    for (int j = 0; j < N; ++j) {

        int dx = i - center;
        int dy = j - center;

        if (dx*dx + dy*dy < radius*radius) {
            T[i * N + j] = 100.0;
        } else {
            T[i * N + j] = 0.0;
        }
    }
}

double cx = ALPHA * DT / (DX * DX);
double cy = ALPHA * DT / (DX * DX);

omp_set_num_threads(num_threads);
omp_set_schedule(sched_type, chunk_size);

auto start =
    std::chrono::high_resolution_clock::now();

for (int step = 0; step < STEPS; ++step) {

    #pragma omp parallel for schedule(runtime)
    for (int i = 1; i < N - 1; ++i) {

        for (int j = 1; j < N - 1; ++j) {

            int idx    = i * N + j;
            int up      = (i - 1) * N + j;
            int down     = (i + 1) * N + j;
            int left     = i * N + (j - 1);
            int right    = i * N + (j + 1);

            T_next[idx] =
                T[idx]
                + cx * (T[up]    - 2*T[idx] + T[down])

```

```

        + cy * (T[left] - 2*T[idx] + T[right]);
    }
}

std::swap(T, T_next);
}

auto end =
    std::chrono::high_resolution_clock::now();

std::chrono::duration<double, std::milli>
    duration = end - start;

double total_energy = 0.0;

#pragma omp parallel for reduction(+:total_energy)
num_threads(num_threads)
    for (int i = 0; i < N * N; ++i) {
        total_energy += T[i];
    }

SimResult res;
res.duration_ms = duration.count();
res.total_energy = total_energy;

return res;
}

int main() {

    std::cout << "With No Threading\n";

    SimResult seq_res =
        run_simulation(1, omp_sched_static, 0);

    std::cout << "Execution time: "
        << std::fixed << std::setprecision(2)
        << seq_res.duration_ms << " ms\n";

    std::cout << "Total Energy: "

```

```

        << std::scientific
        << seq_res.total_energy << "\n";

std::vector<ScheduleConfig> strategies;

strategies.push_back(
    { omp_sched_static, 0, "STATIC" });

strategies.push_back(
    { omp_sched_dynamic, CHUNK_SIZE,
      "DYNAMIC (Chunk " + std::to_string(CHUNK_SIZE) + ")" });

strategies.push_back(
    { omp_sched_guided, CHUNK_SIZE,
      "GUIDED (Chunk " + std::to_string(CHUNK_SIZE) + ")" });

int max_threads = omp_get_max_threads();
if (max_threads < 2) max_threads = 12;

for (int t = 2; t <= max_threads; t += 2) {

    std::cout << "\nWith " << t << " threads\n";

    for (size_t i = 0; i < strategies.size(); ++i) {

        const ScheduleConfig& strat = strategies[i];

        SimResult par_res =
            run_simulation(t,
                          strat.type,
                          strat.chunk_size);

        double speed_up =
            seq_res.duration_ms /
            par_res.duration_ms;

        double ops = (double)N * N * STEPS;

```

```
double throughput =
    ops / (par_res.duration_ms / 1000.0) / 1e6;

double efficiency =
    speed_up / t;

std::cout << "[ "
            << strat.name
            << " ]\n";

std::cout << "Execution time: "
            << std::fixed << std::setprecision(2)
            << par_res.duration_ms << " ms\n";

std::cout << "Speed Up: "
            << speed_up << "x\n";

std::cout << "Throughput: "
            << throughput << "\n";

std::cout << "Efficiency: "
            << efficiency << "\n";
    }
}

return 0;
}
```

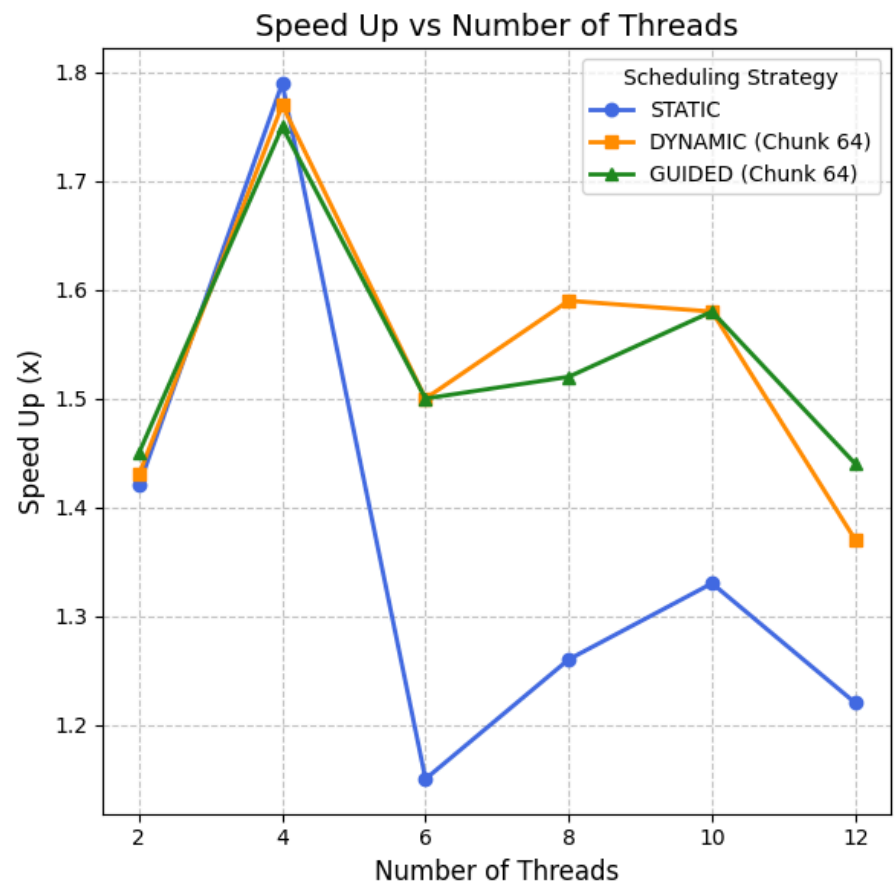
**ON EXECUTION:**

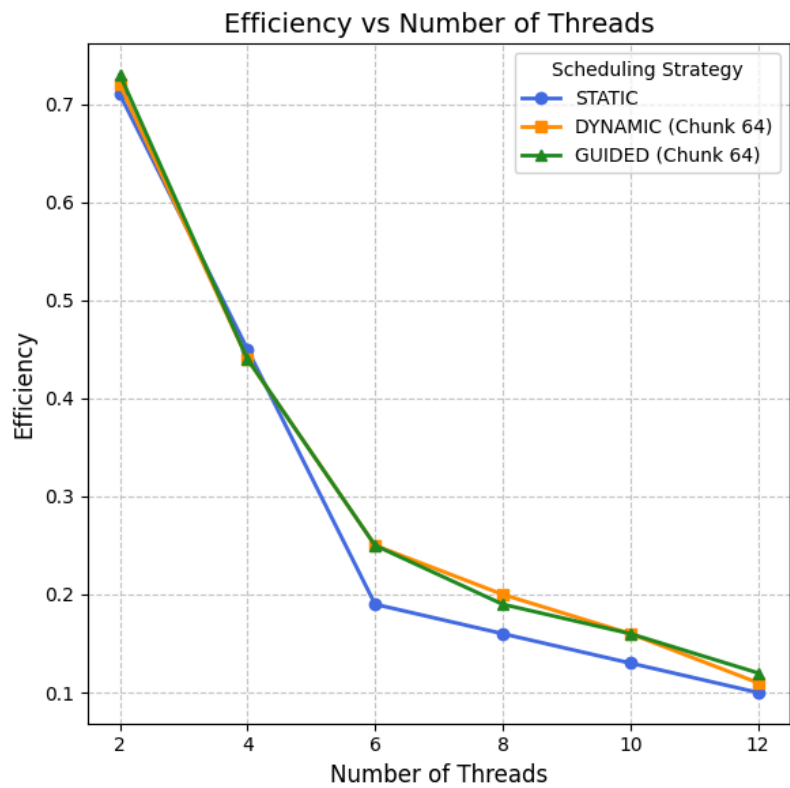
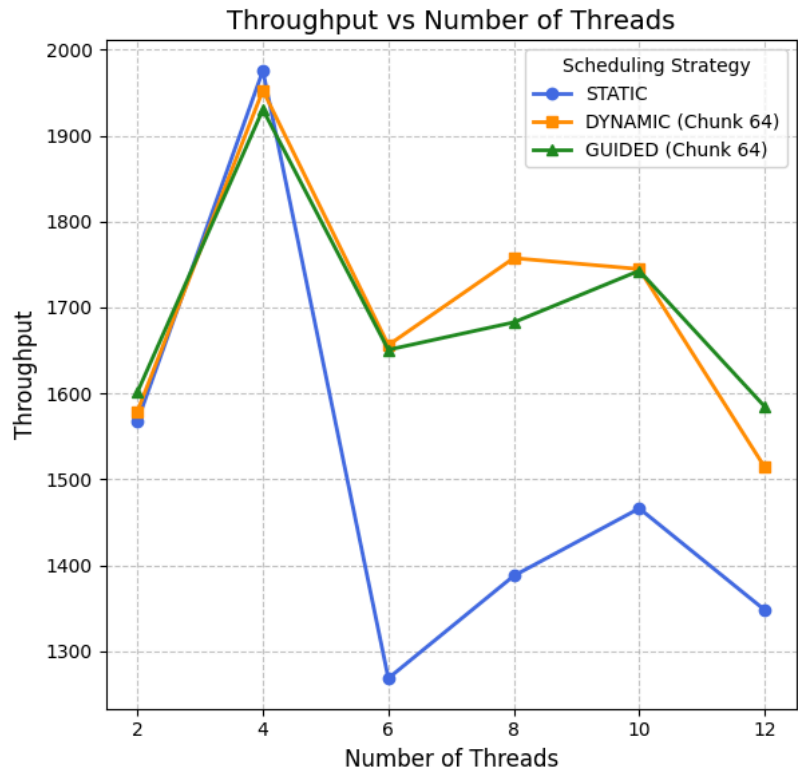
| <b>Number of<br/>Threads</b> | <b>Scheduling<br/>Strategy</b> | <b>Execution<br/>Time (ms)</b> | <b>Speed<br/>Up</b> | <b>Throughput</b> | <b>Efficiency</b> |
|------------------------------|--------------------------------|--------------------------------|---------------------|-------------------|-------------------|
| <b>1</b>                     | <b>N/A</b>                     | <b>1828.93</b>                 | <b>1.00x</b>        | <b>N/A</b>        | <b>N/A</b>        |
| <b>2</b>                     | <b>STATIC</b>                  | <b>1276.87</b>                 | <b>1.43x</b>        | <b>1566.32</b>    | <b>0.72</b>       |
| <b>2</b>                     | <b>DYNAMIC<br/>(Chunk 64)</b>  | <b>1291.53</b>                 | <b>1.42x</b>        | <b>1548.56</b>    | <b>0.71</b>       |
| <b>2</b>                     | <b>GUIDED<br/>(Chunk 64)</b>   | <b>1273.65</b>                 | <b>1.44x</b>        | <b>1570.29</b>    | <b>0.72</b>       |
| <b>4</b>                     | <b>STATIC</b>                  | <b>1126.17</b>                 | <b>1.62x</b>        | <b>1775.92</b>    | <b>0.41</b>       |
| <b>4</b>                     | <b>DYNAMIC<br/>(Chunk 64)</b>  | <b>1070.35</b>                 | <b>1.71x</b>        | <b>1868.55</b>    | <b>0.43</b>       |
| <b>4</b>                     | <b>GUIDED<br/>(Chunk 64)</b>   | <b>1051.49</b>                 | <b>1.74x</b>        | <b>1902.07</b>    | <b>0.43</b>       |
| <b>6</b>                     | <b>STATIC</b>                  | <b>1229.25</b>                 | <b>1.49x</b>        | <b>1627.00</b>    | <b>0.25</b>       |

|    |                       |         |       |         |      |
|----|-----------------------|---------|-------|---------|------|
| 6  | DYNAMIC<br>(Chunk 64) | 1124.51 | 1.63x | 1778.56 | 0.27 |
| 6  | GUIDED<br>(Chunk 64)  | 1165.27 | 1.57x | 1716.35 | 0.26 |
| 8  | STATIC                | 1586.65 | 1.15x | 1260.52 | 0.14 |
| 8  | DYNAMIC<br>(Chunk 64) | 1189.97 | 1.54x | 1680.72 | 0.19 |
| 8  | GUIDED<br>(Chunk 64)  | 1245.05 | 1.47x | 1606.36 | 0.18 |
| 10 | STATIC                | 1446.07 | 1.26x | 1383.06 | 0.13 |
| 10 | DYNAMIC<br>(Chunk 64) | 1183.32 | 1.55x | 1690.16 | 0.15 |
| 10 | GUIDED<br>(Chunk 64)  | 1192.82 | 1.53x | 1676.70 | 0.15 |
| 12 | STATIC                | 1508.26 | 1.21x | 1326.04 | 0.10 |
| 12 | DYNAMIC<br>(Chunk 64) | 1290.93 | 1.42x | 1549.28 | 0.12 |



|    |                      |         |       |         |      |
|----|----------------------|---------|-------|---------|------|
| 12 | GUIDED<br>(Chunk 64) | 1193.56 | 1.53x | 1675.66 | 0.13 |
|----|----------------------|---------|-------|---------|------|





## INFERENCE:

Based on the execution data and the generated graphs for the finite difference method, we can see a very different performance profile compared to previous tasks. The most striking takeaway is how the choice of OpenMP scheduling strategy entirely dictates how the program survives higher thread counts on a hybrid CPU.

- **The Overall Performance Ceiling (Memory Bottleneck):** Unlike the massive speedups seen in purely computational tasks, this heat diffusion simulation hits a hard brick wall. Across all strategies, the **maximum speedup peaks at just 1.74x** (at 4 threads) before degrading. This indicates the algorithm is severely memory-bound. Calculating the next temperature of a grid point requires fetching multiple neighboring points from memory. The CPU's arithmetic units are likely sitting idle, starved for data, because the system's memory bandwidth is fully saturated almost immediately.
- **The 8-Thread Collapse (The Problem with STATIC):** If you look at the **STATIC** scheduling data, there is a massive performance collapse right at **8 threads**. The execution time jumps significantly from **1126ms** (at 4 threads) to **1586ms** (at 8 threads), dropping the speedup to a dismal **1.15x**.
  - **Why it happens:** Static scheduling blindly chops the 2D grid into equal, fixed-size chunks. However, your CPU has a mix of fast Performance (P) cores and slower Efficiency (E) cores. By 8 threads, the system is forced to heavily utilize those slower E-cores. The fast P-cores finish their chunks quickly and then sit entirely idle at the synchronization barrier, waiting for the slower E-cores to finish their massive, equally-sized chunks.
- **The Adaptive Advantage (DYNAMIC & GUIDED):** The **DYNAMIC** and **GUIDED** strategies clearly demonstrate how to mitigate this load imbalance.
  - **Dynamic (Chunk 64):** Instead of one massive chunk, it breaks the work into smaller blocks. It maintains a much steadier execution time (around **1189ms** at 8 threads) compared to Static's spike, as fast cores can grab more work from the queue.
  - **Guided (Chunk 64):** This performs the best overall, achieving the peak speedup of **1.74x** at 4 threads. Crucially, at **12 threads**, it maintains the highest speedup (**1.53x**) compared to Dynamic (**1.42x**) and Static (**1.21x**). By progressively shrinking chunk sizes, it perfectly balances the workload across the P-cores and E-cores, preventing the severe drop-off seen in the static approach.
- **Plunging Efficiency:** Because the memory bus is completely saturated early on, adding more threads simply creates overhead without adding processing power.

By **12 threads**, parallel efficiency collapses to roughly **0.10–0.13**. The system is spending massive amounts of energy (and time managing the threads) for diminishing performance returns.

## OBSERVATION FROM PERF STAT:

```
Performance counter stats for './heat_simulation':
```

|                                   |                         |   |                              |          |
|-----------------------------------|-------------------------|---|------------------------------|----------|
| 161,130.87 msec                   | task-clock              | # | 6.589 CPUs utilized          |          |
| 6,382                             | context-switches        | # | 39.608 /sec                  |          |
| 306                               | cpu-migrations          | # | 1.899 /sec                   |          |
| 31,423                            | page-faults             | # | 195.015 /sec                 |          |
| 472,585,596,312                   | cpu_atom/cycles/        | # | 2.933 GHz                    | (25.47%) |
| 604,163,395,702                   | cpu_core/cycles/        | # | 3.750 GHz                    | (66.03%) |
| 227,592,242,203                   | cpu_atom/instructions/  | # | 0.48 insn per cycle          | (29.72%) |
| 438,722,395,284                   | cpu_core/instructions/  | # | 0.73 insn per cycle          | (66.03%) |
| 13,423,519,755                    | cpu_atom/branches/      | # | 83.308 M/sec                 | (29.73%) |
| 24,100,722,471                    | cpu_core/branches/      | # | 149.572 M/sec                | (66.03%) |
| 14,816,121                        | cpu_atom/branch-misses/ | # | 0.11% of all branches        | (29.73%) |
| 25,486,952                        | cpu_core/branch-misses/ | # | 0.11% of all branches        | (66.03%) |
| TopdownL1 (cpu_core)              |                         | # | 78.7 % tma_backend_bound     |          |
|                                   |                         | # | 0.9 % tma_bad_speculation    |          |
|                                   |                         | # | 2.7 % tma_frontend_bound     |          |
|                                   |                         | # | 17.7 % tma_retiring          | (66.03%) |
| TopdownL1 (cpu_atom)              |                         | # | 1.9 % tma_bad_speculation    |          |
|                                   |                         | # | 13.5 % tma_retiring          | (29.72%) |
|                                   |                         | # | 74.5 % tma_backend_bound     |          |
|                                   |                         | # | 74.5 % tma_backend_bound_aux |          |
|                                   |                         | # | 10.1 % tma_frontend_bound    | (29.72%) |
| 24.456072194 seconds time elapsed |                         |   |                              |          |
| 160.711769000 seconds user        |                         |   |                              |          |
| 0.380247000 seconds sys           |                         |   |                              |          |

### 1. Resource Utilization:

- **Parallel Efficiency:** With **6.589** CPUs utilized, the system is moderately parallelized, distributing the **161.13** seconds of task-clock time across roughly **6-7** processor cores simultaneously during the **24.46**-second elapsed physical time.
- **Hybrid Execution:** The workload is actively split between the Performance-cores (running at **3.750** GHz) and Efficiency/Atom-cores (running at **2.933** GHz).

### 2. Instruction Efficiency:

- **P-Core Performance:** The Performance cores show a low IPC (Instructions Per Cycle) of **0.73**. Only **17.7%** of operations are successfully retiring. This indicates the fast cores are spending a significant portion of their time stalled.
- **E-Core Lag:** The Efficiency (Atom) cores perform similarly with an IPC of **0.48** and a retiring rate of **13.5%**.

- **Branch Prediction:** Unlike the DNA sequence alignment, both core types handle branch prediction exceptionally well. Branch misses are practically non-existent at **0.11%** for P-cores and **0.11%** for E-cores. The finite difference loops used in heat simulations are highly regular, so the CPU almost always guesses the control flow correctly.

### 3. Primary Bottlenecks:

- **Severe Backend Bound (78.7% on P-Cores, 74.5% on Atom):** This is the defining characteristic of this heat simulation. Both core types are heavily choked in the backend. With roughly **three-quarters** of the execution pipeline stalled on both the fast and slow cores, the processing units are starved for data. They are spending most of their time waiting on memory access (RAM) to fetch the 2D grid neighbors for the stencil computation, proving that the application is limited by memory bandwidth. Because the loops are so predictable, time wasted on bad speculation is negligible (**0.9%** on P-cores and **1.9%** on E-cores). The CPU is rarely throwing away bad work; it is simply forced to wait for data to arrive.